

GAME PROGRAMMING COURSE — C# BASICS

構造体入門

Paiza 演習で身につける C# の基礎

対象：1年生（2次元配列まで既習）

STRUC
T

本日の目標

- 構造体（struct）の基本を理解し、自分で書けるようになる
- 配列だけでは管理しづらいデータを、構造体でまとめて扱えるようになる
- コンストラクタを使って、初期化をすっきり書けるようになる
- Paiza の問題演習を通して、実際に手を動かしながら身につける

POINT

この後のページでは、コマごとの目標（このコマの目標）も個別に示します。

LESSON 1

構造体の基礎

目標：構造体の基本を理解し、Paiza の問題を解けるようになる



このコマの目標

- 配列だけでデータを管理する不便さを体感する
- 構造体（struct）が何のためにあるかを理解する
- 構造体を定義し、インスタンス化して使えるようになる
- Paiza の問題を一緒に解き、実際に手を動かす

POINT

このコマのゴールは「なぜ構造体が必要か」を体感し、基本の書き方に慣れることです。

配列だけで管理すると、こうなる

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         string[] names = { "ゆうた", "みさき", "けん" };
9         int[] hps = { 100, 80, 120 };
10        int[] attacks = { 15, 10, 20 };
11
12        Console.WriteLine(names[0]);
13        Console.WriteLine(hps[0]);
14        Console.WriteLine(attacks[0]);
15    }
16 }
```

注意

「名前・HP・攻撃力」は本来ワンセットなのに、3本の配列に分かれてしまっている。

構造体（struct）とは

- 複数の変数（データ）を1つにまとめる「型」。int や string と同じ感覚で使える
- まとめた変数のことを「フィールド」と呼ぶ → public struct 型名 { フィールドの並び }

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 public struct PlayerData
5 {
6     public string Name;
7     public int Hp;
8 }
9 class Program
10 {
11     static void Main()
12     {
13     }
14 }
```

インスタンス化とは

- 構造体は「設計図」。設計図から実際のデータ（実体）を作ることを「インスタンス化」と呼ぶ
- `new 型名 ();` と書くことで、設計図をもとに実体が 1 つ作られる
- 作った実体は、変数に入れて名前をつけて扱う（例： `p1` ）
 - － `PlayerData p1 = new PlayerData();` → 「 `PlayerData` 型の実体を作り、 `p1` という名前で使う」という意味

構造体を書いてみる（インスタンス化）

c# Program.cs

```
1 public struct PlayerData
2 {
3     public string Name;
4     public int Hp;
5 }
6 class Program
7 {
8     static void Main()
9     {
10         PlayerData p1 = new PlayerData();
11         p1.Name = " ゆうた ";
12         p1.Hp = 100;
13         Console.WriteLine(p1.Name);
14         Console.WriteLine(p1.Hp);
15     }
16 }
```


複数のインスタンスをまとめて扱う

c# Program.cs

```
1 public struct PlayerData
2 {
3     public string Name;
4     public int Hp;
5 }
6 class Program
7 {
8     static void Main()
9     {
10         PlayerData p1 = new PlayerData();
11         p1.Name = "ゆうた";
12         p1.Hp = 100;
13         PlayerData p2 = new PlayerData();
14         p2.Name = "みさき";
15         p2.Hp = 80;
16         Console.WriteLine(p1.Name + " " + p1.Hp);
17         Console.WriteLine(p2.Name + " " + p2.Hp);
18     }
19 }
```

配列で構造体を扱う

c# Program.cs

```
1 public struct PlayerData
2 {
3     public string Name;
4     public int Hp;
5 }
6 class Program
7 {
8     static void Main()
9     {
10         PlayerData[] players = new PlayerData[2];
11         players[0].Name = "ゆうた";
12         players[0].Hp = 100;
13         players[1].Name = "みさき";
14         players[1].Hp = 80;
15         foreach (PlayerData p in players)
16         {
17             Console.WriteLine(p.Name + " " + p.Hp);
18         }
19     }
20 }
```

このコマのポイント

- 構造体は「複数のデータを 1 つにまとめる型」
- `public struct 型名 { フィールドの並び }` で定義する
- `変数名 . フィールド名` で値の代入・取得ができる
- `new 型名 ();` でインスタンス（実体）を作る
- `PlayerData[]` のように、構造体は配列にもできる

Unity にもある構造体

- 今日学んでいる構造体は、実は Unity の中でもたくさん使われている
- 位置や向きを表す `Vector3`、色を表す `Color` など構造体（struct）
- `Color` も同じように `r・g・b・a` という4つの値をまとめた構造体
- 「複数の値をひとまとめにする」という考え方は、今日の `PlayerData` と同じ

c# Unity (イメージ)

```
1 // Vector3 のイメージ (簡略化)
2 public struct Vector3
3 {
4     public float x;
5     public float y;
6     public float z;
7 }
```

演習へ： Paiza の問題を一緒に解く

- ここからはこのコマで学んだ構造体の書き方を使って、実際に Paiza の問題を解いていきます。



構造体の更新

https://paiza.jp/works/mondai/class_primer/class_primer__change/edit?language_uid=c-sharp

POINT

分からなくなったら、まず「どのデータをまとめたいか」をノートにまとめてみましょう。

入力を構造体で受け取る（全体像）

c# Program.cs

```
1 public struct PersonalData
2 {
3     public string name;
4     public string old;
5     public string birthday;
6     public string hometown;
7 }
8 class Program
9 {
10     static void Main()
11     {
12         string text = Console.ReadLine();
13         int[] numbers = text.Split(" ").Select(int.Parse).ToArray();
14         int baseLength = numbers[0];
15         int changeLength = numbers[1];
16         PersonalData[] datas = new PersonalData[baseLength];
17         for(int i = 0; i < baseLength; i++)
18         {
19             // 1件ずつ受け取る処理（次のページで詳しく見る）
20         }
21     }
22 }
```

入力を構造体で受け取る（1件ずつのループ）

- for の中身を拡大すると、1行読み取っては構造体に詰める処理を繰り返しているだけ、と分かります。

C# Program.cs

```
1  for(int i = 0; i < baseLength;i++)
2  {
3      text = Console.ReadLine();
4      string[] split = text.Split(" ");
5      datas[i].name = split[0];
6      datas[i].old = split[1];
7      datas[i].birthday = split[2];
8      datas[i].hometown = split[3];
9  }
```

LESSON 2

コンストラクタとリファクタリング

目標：コンストラクタを理解し、コードをリファクタリングできるようになる

2

前回までの振り返り

- 構造体＝複数のデータをまとめる型
- `PlayerData p1 = new PlayerData();` のあと、フィールドに 1 つずつ代入していた
- → フィールドが増えるほど、代入する行も増えていく

POINT

`new` する時点で、まとめて値を渡せたら楽になりそうですね。

コンストラクタとは

- インスタンスが作られる瞬間に自動で呼ばれる特別な処理。構造体名と同じ名前で書く
- 引数で受け取った値を、そのままフィールドに設定できる

c# Program.cs

```
1 public struct PlayerData
2 {
3     public string Name;
4     public int Hp;
5
6     public PlayerData(string name, int hp)
7     {
8         Name = name;
9         Hp = hp;
10    }
11 }
```

コンストラクタあり／なしを比べる

c# Before

```
1 // --- コンストラクタなし ---  
2 PlayerData p1 = new PlayerData();  
3 p1.Name = " ゆうた ";  
4 p1.Hp = 100;
```

c# After

```
1 // --- コンストラクタあり ---  
2 PlayerData p1 = new PlayerData(" ゆうた ", 100);  
3  
4 // new する時点で1行にまとめられる
```

POINT

行数が減るだけでなく、フィールドの代入し忘れも防げます。

リファクタリングの考え方

- リファクタリング＝動きを変えずに、コードを書き直して整理すること
- ① 構造体の中に、フィールドを初期化するコンストラクタを追加する
- ② Main 側の「new → 1 行ずつ代入」を「new(値 , 値 , ...)」の 1 行にまとめる
- ③ 実行結果が前と変わっていないか確認する

演習： 説いた問題をリファクタリングする

- 前回までに解いた「構造体の更新」を開き、コンストラクタを使った形に書き直してみましょう。



構造体の更新（前回の続き）

https://paiza.jp/works/mondai/class_primer/class_primer__change/edit?language_uid=c-sharp

注意

書き直したら、必ず実行して結果が変わっていないことを確認しましょう。

このコマのまとめ

- コンストラクタ = new した瞬間に自動実行される初期化処理
- 引数でまとめて値を受け取り、フィールドに設定できる
- リファクタリングで、同じ動きのコードをより整理された形にできた

できるようになったこと

- コンストラクタの定義
- 1行での初期化
- 既存コードのリファクタリング

次回（ LESSON 3 ）

- 類題 2 問に挑戦する
- 自分の書き方を定着させる

LESSON 3

類題演習と答え合わせ

目標：類題演習を通して構造体の書き方を定着させる



前回の振り返り

- **コンストラクタ = new した瞬間に自動実行される初期化处理**
- `public PlayerData(string name, int hp) { Name = name; Hp = hp; }`
- `PlayerData p1 = new PlayerData(" ゆうた ", 100);` のように 1 行で初期化できる

このコマの進め方

- ① 類題を 2 問、各自のペースで解く
- これまでの構造体の書き方を参考に、似た形の問題に挑戦する
- ② 答え合わせをする
- つまづいたポイントや別の書き方をみんなで共有する

演習：類題に挑戦



文字と整数の組のソート 1

https://paiza.jp/works/mondai/prob60/sort_asc_9/edit?language_uid=c-sharp



文字と整数の組のソート 2

https://paiza.jp/works/mondai/prob60/sort_add_9/edit?language_uid=c-sharp

POINT

1 問目が解けたら、2 問目は自力で挑戦してみましょう。

このコマのまとめ

- 構造体の書き方に、演習を通して慣れてきた
- 似た形の問題でも、フィールドの設計から考える練習になった

できるようになったこと

- struct の定義を自力で書く
- 複数フィールドの使い分け
- 自分のミスに気づいて直す

次回（ LESSON 4 ）

- 各自のペースで自習
- スキルチェック問題に挑戦

LESSON 4

自習：スキルチェック

目標：スキルチェック問題に自力で取り組めるようになる



このコマの進め方

- このコマは自習の時間です。各自のペースで問題に取り組みましょう。
- 分からないところがあれば、遠慮なく質問してください
- 早く終わった人は、別の書き方がないか見直してみましょう

スキルチェック問題

- 基本は Paiza のスキルチェックに挑戦しましょう。



Paiza スキルチェック

おすすめ

<https://paiza.jp/challenges>

- この授業に関連した問題として、STEP_UP 問題（静的メンバ）もあります。



このコマの STEP_UP 問題：静的メンバ C# 編

https://paiza.jp/works/mondai/class_primer/class_primer__static_member/edit?language_uid=c-sharp

- 何をすればいいか迷う人・自信がない人は、まず配列の復習から取り組んでみましょう。



配列の復習：宝くじ C# 編

https://paiza.jp/works/mondai/c_rank_skillcheck_archive/lottery/edit?language_uid=c-sharp



配列の復習：みかんの仕分け C# 編

https://paiza.jp/works/mondai/c_rank_skillcheck_archive/mikan/edit?language_uid=c-sharp

これまでの振り返り

- LESSON 1： 構造体の基礎を学び、 Paiza の問題を一緒に解いた
- LESSON 2： コンストラクタを学び、コードをリファクタリングした
- LESSON 3： 類題を解いて、書き方を定着させた
- LESSON 4： 自習でスキルチェック問題に挑戦した

お疲れさまでした

構造体の考え方は、これから学ぶクラスや Unity での
キャラクター管理にもつながる大切な土台です。